

ICS 344 - Information Security– Assignment 2

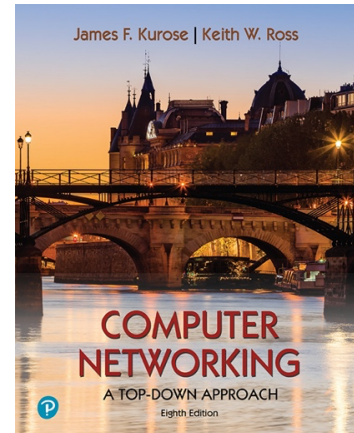
Due: April 24, 2026

Wireshark Lab: TLS

Supplement to *Computer Networking: A Top-Down Approach*, 8th ed., J.F. Kurose and K.W. Ross

“Tell me and I forget. Show me and I remember. Involve me and I understand.” Chinese proverb

© 2005-2021, J.F Kurose and K.W. Ross, All Rights Reserved



In this lab, we'll investigate Transport Layer Security (known as TLS) and aspects of the authentication, data integrity, and confidentiality services provided by TLS. TLS is the successor to the now-deprecated Secure Sockets Layer (SSL).

We'll investigate TLS by analyzing a Wireshark packet trace captured during the retrieval of a web page via HTTPS - a secure version of HTTP, which implements TLS on top of HTTP. We'll look at TLS's client-server handshaking protocol in some detail, since that's where most of the interesting action happens. You may want to review useful online resources for learning more about TLS are [here](#)¹, [here](#)², and [here](#)³; and, of course, in [RFC 5246](#).

1. Capturing packets in an TLS session

The first step in this lab is to capture the packets in an TLS session. To do this, you should startup Wireshark and begin packet capture, retrieve the homepage from <https://www.cics.umass.edu> using the browser of your choice, and then stop Wireshark packet capture. The 's' after 'http' will cause the **Hypertext Transfer Protocol Secure**

¹ “What is Transport Layer Security (TLS)?”, Cloudflare, <https://www.cloudflare.com/learning/ssl/transport-layer-security-tls/>

² A. Munshi, “TLS v1.2 handshake overview”, <https://medium.com/@ethicalevil/tls-handshake-protocol-overview-a39e8eee2cf5>

³ “Taking a Closer Look at the SSL/TLS Handshake,” <https://www.thesslstore.com/blog/explaining-ssl-handshake/#the-tls-12-handshake-step-by-step>

(HTTPS) - an extension of HTTP – to be used to securely retrieve the homepage from www.cics.umass.edu. Here, “securely” means that the www.cics.umass.edu server will be authenticated by your web browser, that the transmission of your client HTTP GET request and the server’s reply will be encrypted, and the integrity of all message content will be cryptographically verified. Of course, the authentication, integrity and encryption of a computer science department’s webpage may not be as critical as that for Internet commerce and banking sites, but the same TLS protocol and TLS messages are used in all cases.

2. A first look at the captured trace

Let’s first set Wireshark’s display so that only the packets to and from www.cics.umass.edu, whose IP address is 34.227.156.202, are displayed. To do this, enter

```
ip.addr == 34.227.156.202
```

in Wireshark’s display filter window. Your screen should look similar to what is shown in Figure 1.

Please note that the IP address may not be exactly 34.227.156.202.

ip.addr == 34.227.156.202						
No.	Time	Source	Destination	Protocol	Length	Info
172	2.564392	192.168.100.6	34.227.156.202	TCP	78	64149 → 443 [SYN] Seq=0 Win=65535 Len=0 MSS=1460 WS=0
213	2.736072	34.227.156.202	192.168.100.6	TCP	74	443 → 64149 [SYN, ACK] Seq=0 Ack=1 Win=26847 Len=0 MSS=1460
214	2.736144	192.168.100.6	34.227.156.202	TCP	66	64149 → 443 [ACK] Seq=1 Ack=1 Win=131584 Len=0 TSval=1466
215	2.736552	192.168.100.6	34.227.156.202	TCP	1466	64149 → 443 [ACK] Seq=1 Ack=1 Win=131584 Len=1400 TSval=1466
216	2.736553	192.168.100.6	34.227.156.202	TLSv1...	650	Client Hello (SNI=www.cics.umass.edu)
228	2.910404	34.227.156.202	192.168.100.6	TCP	66	443 → 64149 [ACK] Seq=1 Ack=1401 Win=29696 Len=0 TSval=1466
229	2.910549	34.227.156.202	192.168.100.6	TCP	66	443 → 64149 [ACK] Seq=1 Ack=1985 Win=32768 Len=0 TSval=1466
230	2.911425	34.227.156.202	192.168.100.6	TLSv1...	1466	Server Hello
231	2.911671	34.227.156.202	192.168.100.6	TCP	1466	443 → 64149 [ACK] Seq=1401 Ack=1985 Win=32768 Len=0 TSval=1466
232	2.911720	192.168.100.6	34.227.156.202	TCP	66	64149 → 443 [ACK] Seq=1985 Ack=2801 Win=129664 Len=0 TSval=1466
233	2.912038	34.227.156.202	192.168.100.6	TCP	1466	443 → 64149 [ACK] Seq=2801 Ack=1985 Win=32768 Len=1400 TSval=1466
234	2.912039	34.227.156.202	192.168.100.6	TLSv1...	1108	Certificate, Server Key Exchange, Server Hello Done
235	2.912069	192.168.100.6	34.227.156.202	TCP	66	64149 → 443 [ACK] Seq=1985 Ack=5243 Win=128576 Len=0 TSval=1466
236	2.912539	192.168.100.6	34.227.156.202	TLSv1...	192	Client Key Exchange, Change Cipher Spec, Encrypted Handshake
237	2.912656	192.168.100.6	34.227.156.202	TLSv1...	165	Application Data
238	2.912804	192.168.100.6	34.227.156.202	TLSv1...	1242	Application Data
247	3.083227	34.227.156.202	192.168.100.6	TLSv1...	324	New Session Ticket, Change Cipher Spec, Encrypted Handshake
248	3.083303	192.168.100.6	34.227.156.202	TCP	66	64149 → 443 [ACK] Seq=3386 Ack=5501 Win=130752 Len=0 TSval=1466
249	3.083505	34.227.156.202	192.168.100.6	TLSv1...	135	Application Data
250	3.083539	192.168.100.6	34.227.156.202	TCP	66	64149 → 443 [ACK] Seq=3386 Ack=5570 Win=130944 Len=0 TSval=1466
251	3.083644	192.168.100.6	34.227.156.202	TLSv1...	104	Application Data
252	3.083790	34.227.156.202	192.168.100.6	TLSv1...	104	Application Data
253	3.083823	192.168.100.6	34.227.156.202	TCP	66	64149 → 443 [ACK] Seq=3424 Ack=5608 Win=131008 Len=0 TSval=1466
254	3.085070	34.227.156.202	192.168.100.6	TLSv1...	565	Application Data
255	3.085103	192.168.100.6	34.227.156.202	TCP	66	64149 → 443 [ACK] Seq=3424 Ack=6107 Win=130560 Len=0 TSval=1466
256	3.085364	192.168.100.6	34.227.156.202	TLSv1...	108	Application Data
306	3.232488	192.168.100.6	34.227.156.202	TLSv1...	428	Application Data
309	3.255210	34.227.156.202	192.168.100.6	TCP	66	443 → 64149 [ACK] Seq=6107 Ack=3466 Win=35328 Len=0 TSval=1466
474	3.406334	34.227.156.202	192.168.100.6	TLSv1...	560	Application Data
475	3.406401	192.168.100.6	34.227.156.202	TCP	66	64149 → 443 [ACK] Seq=3828 Ack=6601 Win=130560 Len=0 TSval=1466
476	3.406694	192.168.100.6	34.227.156.202	TLSv1...	108	Application Data

Figure 1: Wireshark display showing TCP and TLS message to/from 34.227.156.202

It's important to keep in mind that an Ethernet frame (containing an IP datagram containing an TCP segment) may contain one or more TLS records. (This is very different from HTTP, for which each frame contains either one complete HTTP message

or a portion of a HTTP message.) Also, a TLS record may not completely fit into an Ethernet frame, in which case multiple frames will be needed to carry the record.

In answering the questions below, please use your own live trace and supplement your answer with screenshots.

We've said earlier that HTTPS implements TLS running "over" TCP. That means that a TCP connection must first be established between your browser and the web server for www.cics.umass.edu before TLS and HTTP messages can be exchanged, just as we saw with the vanilla (non-TLS) HTTP protocol.

1. What is the packet number in your trace that contains the initial TCP SYN message? (By "packet number," we meant the number in the "No." column at the left of the Wireshark display, not the sequence number in the TCP segment itself).
2. Is the TCP connection set up before or after the first TLS message is sent from client to server?

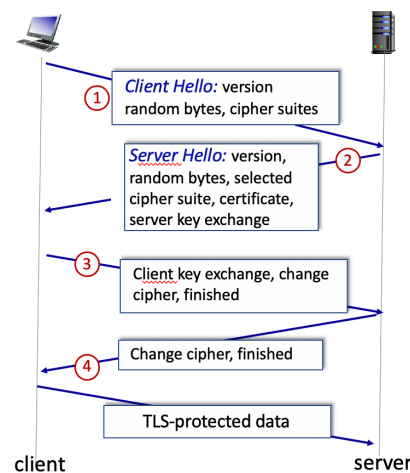


Figure 2: TLS handshaking

3. The TLS Handshake: *Client Hello* message

We learned that, as shown in Figure 2, the client-server TLS handshake begins with the client sending a TLS *Client Hello* message.

3. What is the packet number in your trace that contains the TLS *Client Hello* message?
4. What version of TLS is your client running, as declared in the *Client Hello* message?
5. How many cipher suites are supported by your client, as declared in the *Client Hello* message? A cipher suite is a set of related cryptographic algorithms that

determine how session keys will be derived, and how data will be encrypted and be digitally signed via a HMAC algorithm.

6. Your client generates and sends a type of nonce as a string of “random bytes” to the server in the *Client Hello* message. What are the first two hexadecimal digits in the random bytes field of the *Client Hello* message? Enter the two hexadecimal digits (without spaces between the hex digits and without any leading '0x' , using lowercase letters where needed). *Hint:* be careful to fully dig into the Random field to find the Random Bytes subfield (do not consider the GMT UNIX Time subfield of Random).
7. What is the purpose(s) of the “random bytes” field in the *Client Hello* message? Note: you’ll have to do some searching and reading to get the answer to this question; see section 8.6 and in [RFC 5246](#) (section 8.1 in RFC 5246 in particular).

3. The TLS Handshake: *Server Hello* message

Next, let’s take a look at the second step of the TLS handshake, the TLS *Server Hello* message, which is sent in response to the earlier TLS *Client Hello* message.

8. What is the packet number in your trace that contains the TLS *Server Hello* message?
9. Which cipher suite has been chosen by the server from among those offered in the earlier *Client Hello* message?
10. Does the *Server Hello* message contain random bytes, similar to how the *Client Hello* message contained random bytes? And if so, what is/are their purpose(s)?

In response to the initial *Client Hello* message, the server also sends back two important additional pieces of information (that may be contained in a different packet than the initial part of the *Server Hello* message. The first important piece of information returned is a certificate issued by a trusted certification authority (CA; see section 8.3 in the text) that binds the public key (and other information) of the web server to that web server’s identity. Your client may use the server’s public key for a number of different purposes, including deriving the symmetric keys to encrypt data being sent over this HTTPS/TLS/TCP session and for generating HMAC digital signatures. You can, of course, look at the server’s certificate in Wireshark. You can also view the server’s certificate in your web browser after the [www.cs.umass.edu](#) server has responded to your request (and the certificate formatted a lot prettier too) – [here’s how](#).

The second piece of additional information returned from the server are parameters needed for the symmetric encryption of the data (HTTP messages in this case) being exchanged.

Let’s dig a bit deeper into the public key certificate.

11. What is the packet number in your trace for the TLS message part that contains the public key certificate for the [www.cics.umass.edu](#) server (actually the [www.cs.umass.edu](#) server)?

12. A server may return more than one certificate. If more than one certificate is returned, are all of these certificates for `www.cs.umass.edu`? If not all are for `www.cs.umass.edu`, then who *are* these other certificates for? You can determine who the certificate is for by checking the `id-at-commonName` or (`Common Name`) field in the returned certificate.
13. What is the name of the certification authority that issued the certificate for `id-at-commonName=www.cs.umass.edu`?
14. What digital signature algorithm is used by the CA to sign this certificate? Hint: this information can be found in `signature` subfield of the `SignedCertificate` field of the certificate for `www.cs.umass.edu`.
15. Let's take a look at what a real public key looks like! What are the first four hexadecimal digits of the modulus of the public key being used by `www.cics.umass.edu`? Enter the four hexadecimal digits (without spaces between the hex digits and without any leading '0x', using lowercase letters where needed, and including any leading 0s after '0x'). Hint: this information can be found in `subjectPublicKeyInfo` subfield of the `SignedCertificate` field of the certificate for `www.cs.umass.edu`.
16. Look in your trace to find messages between the client and a CA to get the CA's public key information, so that the client can verify that the CA-signed certificate sent by the server is indeed valid and has not been forged or altered. Do you see such message in your trace? If so, what is the number in the trace of the first packet sent from your client to the CA? If not, explain why the client did not contact the CA.

The *Server Hello* message is always terminated by an explicit *Server Hello Done* record.

17. What is the packet number in your trace for the TLS message part that contains the *Server Hello Done* TLS record?

4. The TLS Handshake: wrapping up the handshake

After the exchange of *Hello* messages, the client responds to the TLS *Server Hello* message with its own public key information, a declaration (via a *Change Cipher Spec* record) that all further communication will be encrypted via the negotiated algorithm and key, and an *Encrypted Handshake Message* record that contains encrypted information (e.g., a cryptographic hash of all messages exchanged during this handshake) to prevent man-in-the-middle replay attacks.

18. What is the packet number in your trace for the TLS message that contains the public key information, *Change Cipher Spec*, and *Encrypted Handshake* message, being sent from client to server?
19. Does the client provide its own CA-signed public key certificate back to the server? If so, what is the packet number in your trace containing your client's certificate?

Lastly, the server responds to the client with its own a declaration (via a *Change Cipher Spec* record) that all further communication will be encrypted via the negotiated algorithm and key, and an *Encrypted Handshake Message* record which also contains

encrypted information (e.g., a cryptographic hash of all messages exchanged during this handshake) to prevent man-in-the-middle replay attacks.

5. Application data

Once the TLS handshaking has completed, the encrypted application data can begin to flow over the HTTP-over-TLS-over-TCP connection. Of course, since this data is encrypted we can't actually examine the contents of these encrypted messages (and isn't that just the point!). But we can still ask a few last questions about the encrypted traffic.

20. What symmetric key cryptography algorithm is being used by the client and server to encrypt application data (in this case, HTTP messages)?
21. In which of the TLS messages is this symmetric key cryptography algorithm finally decided and declared?
22. What is the packet number in your trace for the first encrypted message carrying application data from client to server?
23. What do you think the content of this encrypted application-data is, given that this trace was generated by fetching the homepage of www.cics.umass.edu?

Lastly, let's take a look at how the client closes the TLS connection. You'll have to dig around a bit to answer this question (see the earlier, footnoted references).

24. What packet number contains the client-to-server TLS message that shuts down the TLS connection? Because TLS messages are encrypted in our Wireshark traces, we can't actually look *inside* a TLS message and so we'll have to make an educated guess here.